ADVANCED PRE-LAB AND LAB ACTIVITY

Multi-Gesture Human–Computer Interaction using OpenCV and MediaPipe

Activity Information	Details
Activity Type	Individual
Pre-Lab	Advanced Gesture Media Controller
Suggested Pre-Lab Duration	2–2.5 hours
Lab	Assigned Advanced HCI Variant
Suggested Lab Duration	3 hours
Surprise Modification	20–30 minutes
Starting Program	Instructor-provided handgesture.py
Core Technologies	Python, OpenCV, MediaPipe, OS/application controls

I. Activity Overview

The instructor-provided handgesture.py is the required starting point. The existing program captures webcam video, detects a hand, extracts MediaPipe landmarks, and uses thumb–index distance to control system volume. Students must progressively extend this program rather than replace it with a separate implementation.

The Southville activity has two major stages. PRE-LAB establishes a common advanced gesture-control foundation for all students. LAB requires each student to transform that foundation into one of ten complex Human–Computer Interaction systems. Existing functionality must remain operational throughout the activity.

II. PRE-LAB — Advanced Gesture Media Controller

Objective

Implement a robust multi-gesture media controller that combines static gesture recognition, motion tracking, distance-based control, state management, confirmation logic, cooldown/debouncing, and an OpenCV dashboard.

Required Pre-Lab Functions

Control	Required Gesture / Logic	Required Function
Volume	Thumb–Index distance	Continuous volume increase/decrease with percentage indicator.
Play/Pause	Open palm	Toggle current media playback.
Next Track	Swipe right	Use frame-to-frame horizontal motion tracking.
Previous Track	Swipe left	Use frame-to-frame horizontal motion tracking.
Mute/Unmute	Closed fist	Toggle audio mute state.
Lock/Unlock	Two-finger gesture	Enable or disable command execution without closing the application.
Confirmation	Gesture hold	Static commands must satisfy a confirmation threshold before execution.
Cooldown	Timed state	Prevent repeated triggering after a command executes.
Invalid Gesture	Unrecognized state	Unknown poses must not execute commands.

Mandatory OpenCV Dashboard

The camera window must continuously show:

- System state: LOCKED / UNLOCKED
- Current gesture
- Current command/action
- Action status: DETECTING / CONFIRMING / EXECUTED / COOLDOWN
- Volume percentage and graphical volume indicator
- Hand landmarks and connections
- Active mode
- Cooldown or confirmation feedback
- Invalid/unrecognized gesture feedback

State-Management Requirement

Students must not implement the controller as a collection of uncontrolled independent actions. The program must maintain a logical state. A recommended sequence is:

LOCKED → UNLOCKED → GESTURE DETECTION → CONFIRMATION → COMMAND EXECUTION → COOLDOWN → GESTURE DETECTION

Motion Detection Requirement

Swipe gestures must be detected using position history across multiple frames. A single-frame hand position is not sufficient. Students must establish an appropriate horizontal displacement threshold and reset or cooldown behavior to avoid multiple track changes from one swipe.

Pre-Lab Deliverables

- Working Python source code based on the instructor-provided file.
- PDF documentation containing algorithm, flowchart, landmark explanation, screenshots, and testing.
- Execution screenshots for every required gesture/state.
- Short demonstration video showing the complete Pre-Lab system.
- Minimum of eight documented test cases, including invalid gestures and repeated-trigger prevention.

III. LAB — Advanced Individual Variants

Each student receives one assigned variant. The completed Pre-Lab controller remains part of the program. The assigned Lab functions are additions, not replacements. Each final system must support a minimum of eight meaningful gesture-controlled actions across the combined Pre-Lab and Lab implementation.

#	Advanced Variant	Complex Lab Requirements
1	Advanced Spotify Gesture Controller	Create MEDIA and ADVANCED MEDIA modes. Add seek forward/backward, track navigation, playback, volume, mute, controller lock, gesture history, and a visible command timeline. At least one command must require a two-stage Select → Confirm sequence.
2	Gesture-Based Presentation & Annotation System	Add next/previous slide, start/end presentation, pointer mode, freehand annotation, erase, clear annotation, and presentation lock. Implement gesture-based switching between PRESENTATION, POINTER, and ANNOTATION modes.
3	Advanced Virtual Mouse & Navigation System	Add cursor movement, left click, right click, double-click, drag-and-drop, vertical scrolling, and mouse lock/unlock. Clicking and dragging must use different gesture states and accidental clicks must be suppressed.

4	Virtual Whiteboard & Annotation Studio	Implement draw, erase, clear, undo, color selection, brush-size adjustment, line, rectangle, circle, and save-canvas operations. Include a gesture-controlled OpenCV tool palette and visible active-tool state.
5	Computer Vision Multimedia Control Center	Combine media playback, volume, seek, next/previous, fullscreen, mute, and media-mode switching. Add inactivity auto-lock and a gesture sequence to reactivate the controller.
6	Gesture-Based Image Manipulation Workstation	Implement next/previous image, zoom in/out, pan, rotate clockwise/counterclockwise, grayscale, blur, edge view, and reset. Transformations must be shown inside an OpenCV workspace with current operation/state.
7	Dual-Hand Human–Computer Interface	Detect left and right hands separately. Assign different command groups to each hand and implement at least two commands that require coordinated two-hand gestures. Display both hand states independently.
8	Gesture-Based Desktop Productivity Controller	Implement application/window switching, minimize, maximize/restore, copy, paste, undo, redo, screenshot, and scrolling. Include a safety confirmation state for system/window commands and an OpenCV command history.
9	OpenCV Virtual Drawing & Shape Studio	Implement freehand drawing, eraser, color selection, thickness control, line, rectangle, circle, undo, clear, and save output. Tool selection must occur through gesture interaction with an OpenCV toolbar or defined screen zones.
10	Multi-Mode Smart Gesture Control System	Build VOLUME, MEDIA, MOUSE, and PRESENTATION modes. Mode changes must be gesture-controlled and confirmed. Each mode must have its own commands and OpenCV status panel while preventing gestures from one mode from activating commands in another.

IV. Common Complexity Requirements for All 10 Variants

- Retain all working Pre-Lab functions.
- Use at least two static gesture conditions.
- Use at least two motion-based gesture conditions.
- Use at least one distance-based control.
- Implement at least one multi-stage interaction such as Activate → Select → Confirm.
- Use a state or mode variable to prevent command conflicts.
- Implement confirmation and/or cooldown logic for discrete commands.
- Provide visible OpenCV feedback for modes, gestures, actions, and system state.
- Include invalid-gesture handling.
- Demonstrate recovery from an incorrect gesture without restarting the application.
- Students must be able to explain the MediaPipe landmark IDs and logic used for every major gesture.
- Keyboard controls may support testing or emergency exit but may not replace required gesture-controlled functions.

V. Required Testing

Test Category	Minimum Requirement
Pre-Lab Regression	Verify that Volume, Play/Pause, Next, Previous, Mute, and Lock/Unlock still work.
Static Gestures	Test correct recognition and rejection of similar/incorrect poses.
Motion Gestures	Test movement threshold, direction, and repeated-trigger prevention.
Mode/State	Verify commands do not execute in the wrong mode or while locked.
Confirmation	Verify actions execute only after the required confirmation condition.
Cooldown	Verify a single gesture does not cause repeated commands.
Assigned Variant	Test every newly implemented Lab function.
Negative Testing	Use random/invalid poses and confirm that no unintended action occurs.
Recovery	Show that the system returns to a usable state after invalid input.

VI. Live Demonstration

1. Demonstrate all required Pre-Lab controls.
2. Demonstrate lock/unlock and command suppression while locked.
3. Demonstrate the assigned Lab variant and all required new functions.
4. Demonstrate at least one static, one motion-based, one distance-based, and one multi-stage interaction.
5. Demonstrate negative/invalid gesture handling.
6. Demonstrate cooldown or repeated-trigger prevention.
7. Explain selected code sections and landmark IDs when requested by the instructor.
8. Complete the assigned surprise modification and demonstrate it without breaking previous functionality

VII. Documentation Requirements

9. Title Page
10. Objectives
11. Original program analysis
12. Pre-Lab architecture and implemented gestures
13. Assigned Lab variant and requirements
14. MediaPipe landmark map identifying landmarks used
15. System architecture / state diagram
16. Program flowchart
17. Gesture recognition algorithms or pseudocode
18. Explanation of static, motion, distance-based, and multi-stage gestures
19. OpenCV interface/dashboard explanation
20. Modified code sections with technical explanation
21. Test plan and test results
22. Screenshots of all major modes/functions
23. Surprise modification implementation and evidence
24. Reflection: false positives, thresholds, state conflicts, limitations, and proposed improvements

VIII. Grading Rubric — 100 Points

Criterion	Points
Pre-Lab Core Gesture Controller	15
Assigned Lab Variant Functionality	20

Gesture Recognition Accuracy and Robustness	15
State / Mode Management	10
Motion, Distance, and Multi-Stage Gesture Logic	10
OpenCV Interface and Visual Feedback	10
Confirmation, Cooldown, and False-Trigger Prevention	5
Surprise Modification	5
Documentation and Testing	5
Live Code Explanation	5
TOTAL	100

IX. Code Defense / Oral Check

During the live demonstration, the instructor may select any portion of the code and ask the student to explain or modify it. Questions may cover landmark IDs, gesture thresholds, coordinate history, state transitions, cooldown logic, mode conflicts, false-positive prevention, or OpenCV rendering. Students must demonstrate understanding of the submitted implementation, not only its final output.

X. Submission Structure

```
SURNAME_SOUTHVILLE_HANDGESTURE/
├── prelab/
│   ├── handgesture_prelab.py
│   ├── documentation_prelab.pdf
│   └── screenshots/
├── lab/
│   ├── handgesture_lab.py
│   ├── documentation_lab.pdf
│   └── screenshots/
├── surprise_modification/
│   └── final_handgesture.py
└── demo.mp4
```